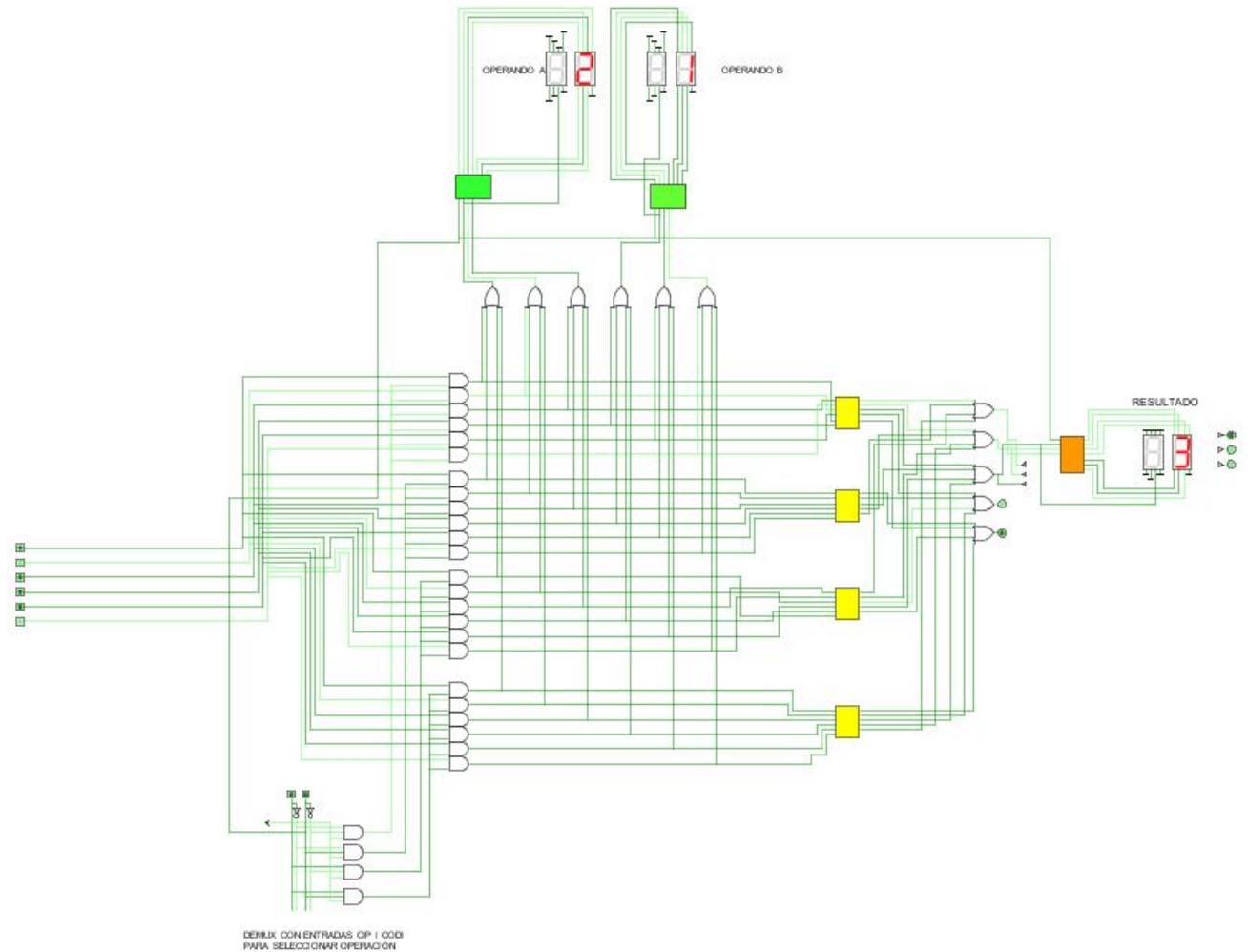


SISTEMES DIGITALS

Pràctica Final Obligatòria Sistema Combinacional



Índex

1.Introducció

2.Descripció i circuit de les parts identificades

2.1 Suma C2

2.2 Suma SM

2.3 Resta C2

2.4 Resta SM

2.5 Display entrades A i B

2.6 Display resultat

2.7 Demultiplexor

3.Joc de proves (parts diferenciades i circuit complet)

3.1 Suma C2

3.2 Suma SM

3.2 Resta C2

3.4 Resta SM

3.5 Display entrades A,B

3.6 Display resultat

3.7 Circuit complet

4.Conclusions

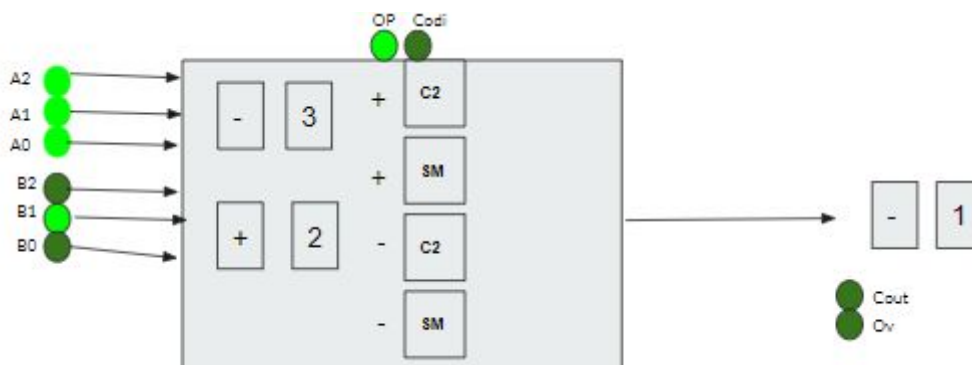
1.Introducció

El nostre objectiu en aquest projecte de sistemes digitals és implementar només amb portes lògiques un circuit que ens faci 4 operacions diferents de 3 bits: suma C2, resta SM, suma SM i resta C2. Això ho duim a terme mitjançant un demultiplexor. El nostre demultiplexor tindrà com a entrades A2,A1,A0 i B2,B1,B0(cada un dels bits de ambdós nombres A i B). Dedins hi haurà les operacions a dur a terme, on es mostraran els operands A i B amb 4 displays, dos per a cada un. Per elegir quina operació haurem de realitzar, tindrem 2 entrades addicionals anomenades operació i codi. El resultat vendrà en funció de la operació elegida.

OpCodi	Operació	Codificació
0 0	$A + B$	Complement a 2
0 1	$A + B$	Signe i Magnitud
1 0	$A - B$	Complement a 2
1 1	$A - B$	Signe i Magnitud

Finalment es mostrarà el resultat en dos displays a la sortida d'aquest demultiplexor, afegint les sortides de carry out y overflow. Cal tenir en compte que el resultat només ve donat per la possibilitat que hi ha en 3 bits.

Esquema visual;



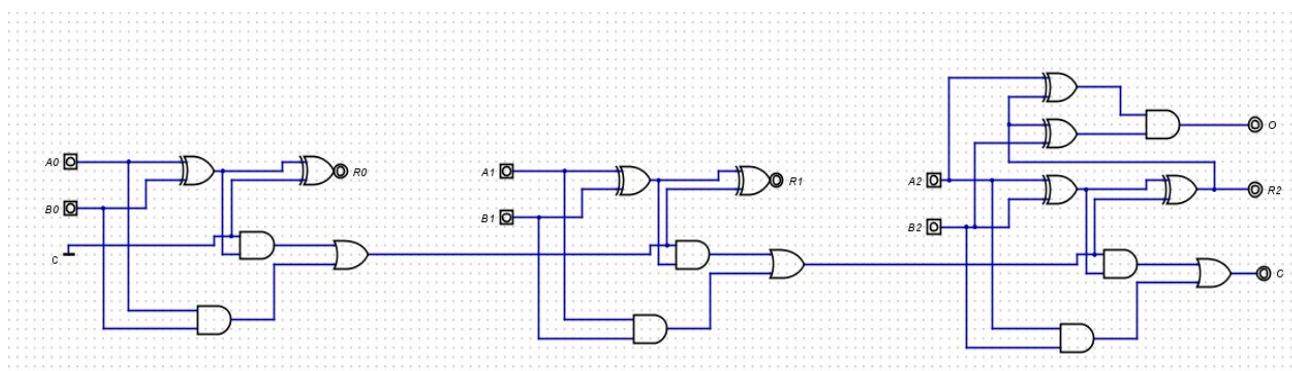
2.Descripció i circuit de les parts identificades

Aquest circuit es divideix ahora en diferents subcircuits, com són: suma C2,suma SM, resta C2,resta SM, display entrada A, display entrada B i display resultat. Anem a veure-los detalladament.

2.1 Suma C2

La primera operació que veiem representada és la suma en C2. Aquest circuit ve estructurada en forma de sumador paral·lel mitjançant la tècnica Ripple Carry Adder, en la qual enganxam en cadena els diferents carry outs que hi han. La funció de R(bit resultat) ve donada per una porta XOR ja que han de ser nombres diferents per a que sigui 1, si fossin iguals seria igual a 0 i tendríem carry out. Per això, la funció de Carry out ve donada per una porta AND(on ambdues entrades han de ser iguals).Per a sumar en C2 es suma igual i l'Overflow estarà present quan els operands siguin del mateix signe i el resultat tenguí el signe contrari.Es comparen els diferents bits de A i B segons la seva posició de significancia, començant desde A0,B0 fins A2,B2. Es suma igual que en binari, tinguent només Overflow en el cas que ambdós tinguin mateix signe i el resultat no.

Aquesta operació és corresponent a la combinació 00 de Op i Codi.



Circuit Suma C2

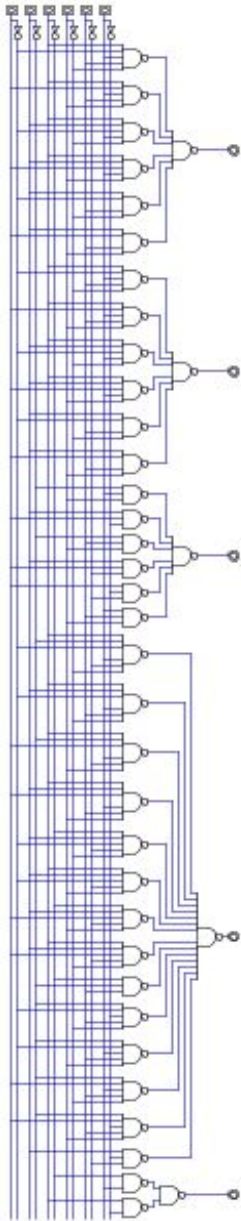
2.2 Suma SM

El segon subcircuit ve a ser en nomenclatura diferent, SM, en el qual canvia el signe segons el bit més significatiu. L'eina digital ens proporciona un gran ajuda amb la possibilitat de crear un circuit mitjançant l'escriptura de la seva respectiva taula de veritat(vegi el punt 3).He fet la seva taula de veritat amb 6 respectives entrades(a0,a1,a2,b2,b1,b0) i sortides (r2,r1,r0,carry out,ov). La suma en SM és més complexa que en C2 ja que també distingim dos casos diferents, quan tenim el mateix signe en els operands o quan tenen diferent signe. Cal recordar que un nombre és negatiu en SM si el bit més significatiu és 1. En el cas que tinguem el mateix signe el procés és més simple, basta conservar el signe i realitzar la suma com si d'una manera normal es tractés. Si el carry out és igual a 1 hi haurà Overflow.

En el cas que tinguem operands amb diferent signe realitzarem un procés molt més complicat que consisteix en escollir la magnitud més gran i restar-li l'altra. Mai hi podrà haver

overflow ja que al ser de diferent signe es restaran, quedant-se el signe de la magnitud més gran escollida. A continuació podeu veure el circuit implementat per únicament portes NAND.

Aquest circuit s'activarà amb la combinació 01 de Op i Codi.



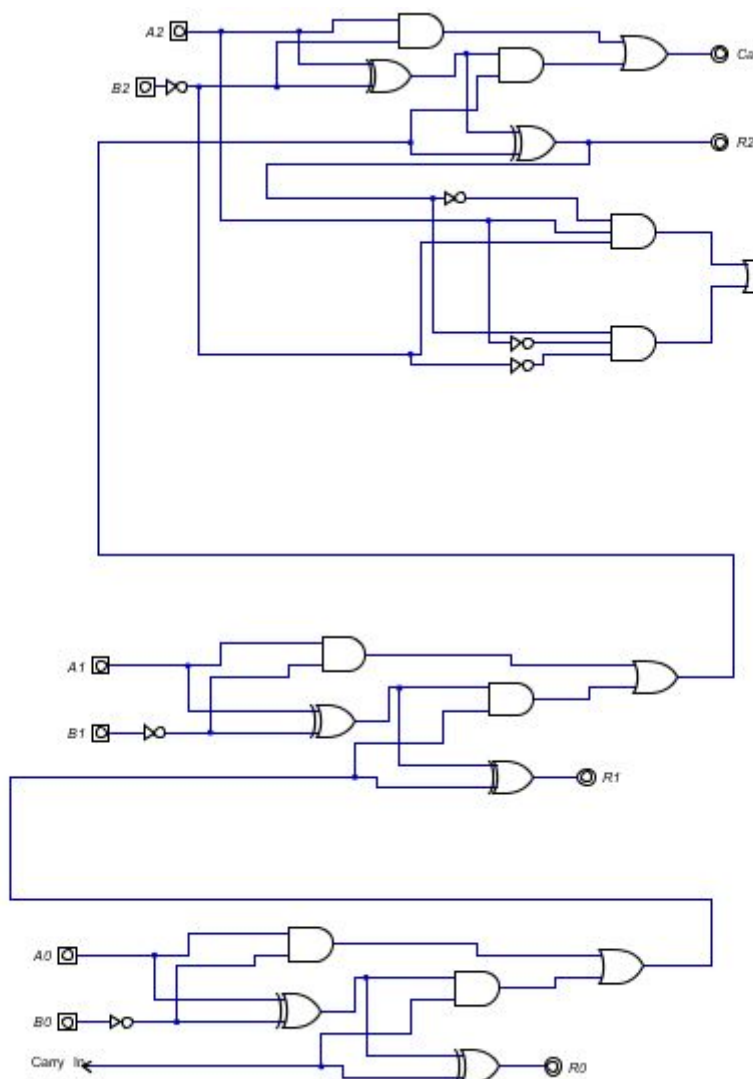
Circuit Suma SM

2.3 Resta C2

La tercera operació a tenir en compte és la resta en C2, circuit que no varia molt de la suma en C2. Per a realitzar la resta en aquest cas hem de transformar la següent expressió $(a-b)$ en $a+(-b)$. Per a dur a terme el canvi de signe en C2 canviarem tots els 1s per 0s i viceversa, afegint al Carry In un 1. La funció del resultat serà la mateixa que en la suma C2, tan sols afegirem portes NOT a les entrades B0, B1 i B2. El resultat seguirà essent donat per una porta XOR i el carry out per una AND entre les dues variables i el Carry In que també es tindria en compte perquè l'hem inicialitzat a 1 per poder realitzar el canvi de signe corresponent a C2.

Com a l'altra operació amb C2, primer es comparen A0,B0 fins a A2,B2. En resum, el circuit sirà el mateix just afegint portes NOT i una entrada constant de 1 al Carry In.

Aquest circuit s'activa amb la parella 10 usant les entrades Op i Codi. Està situat en vertical per tal d'ordenar els bits de més significatiu a menys significatiu.

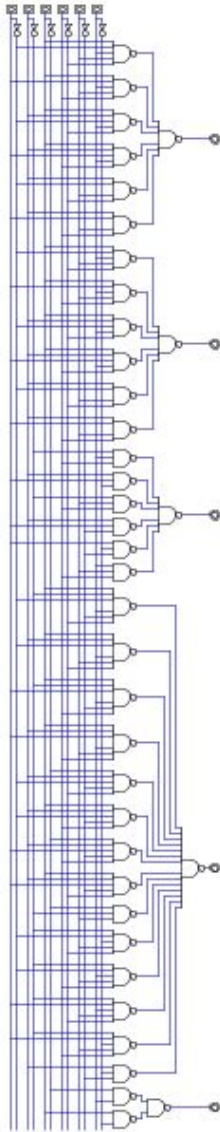


Circuit Resta C2

2.4 Resta SM

El circuit que implementa l'acció de restar en SM vendria a ser el mateix que abans. Com pot ser possible? Per a restar, intentem aconseguir una expressió en forma de suma, a partir de $(a-b)$ fins a arribar a $(a+(-b))$. Aconseguirem això implementant un sumador de la mateixa manera que abans, amb la diferència de signe del segon operand. És a dir, canviem el signe i els sumam. Si la sortida carry out és 1, implica que tenim present overflow. En aquest subcircuit també he fet la taula de veritat a mà i l'he sintetitzada amb l'eina digital, circuit creat amb portes NAND.

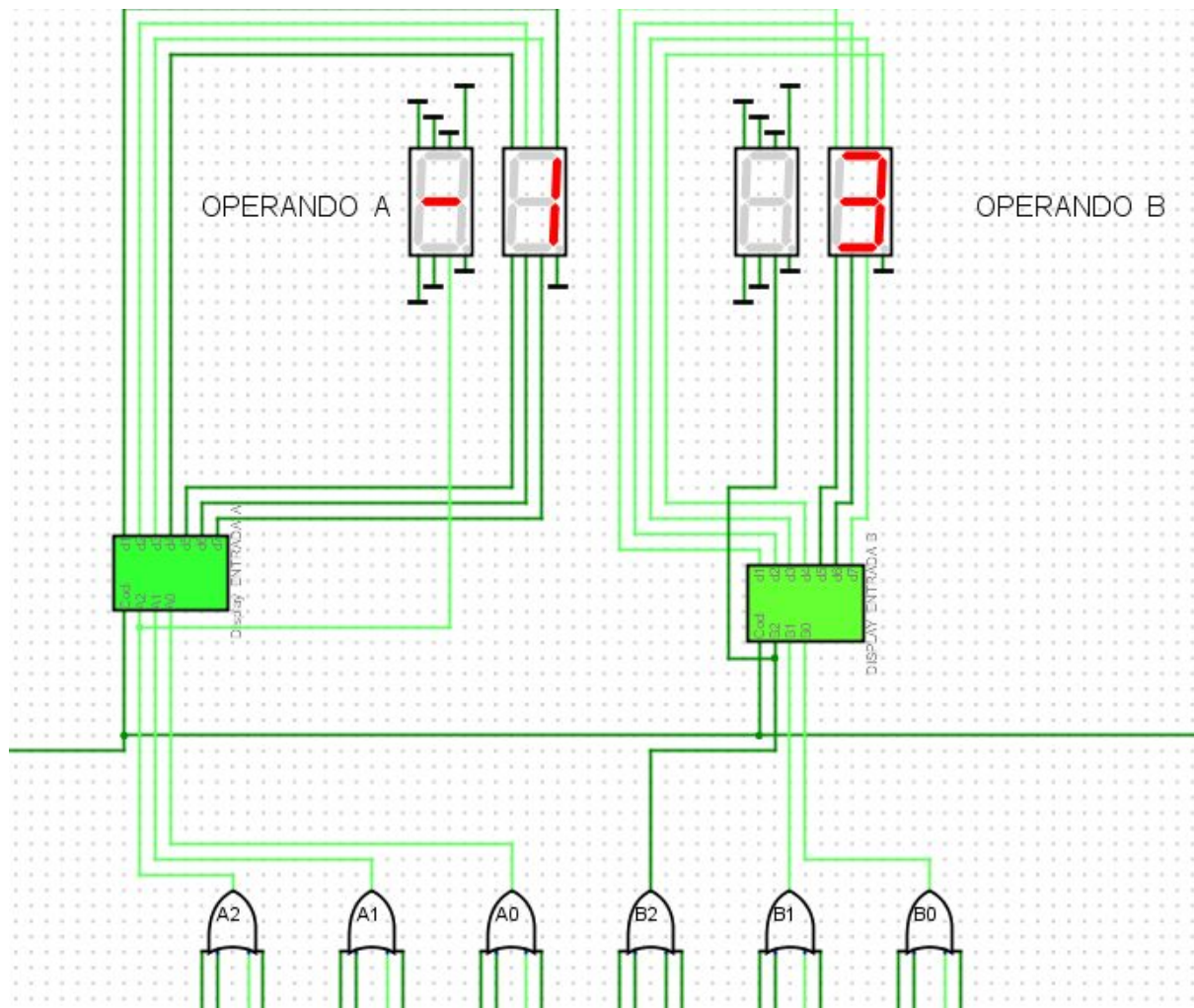
Aquesta darrera operació possible s'activa mitjançant la sortida 11 de Op i Codi



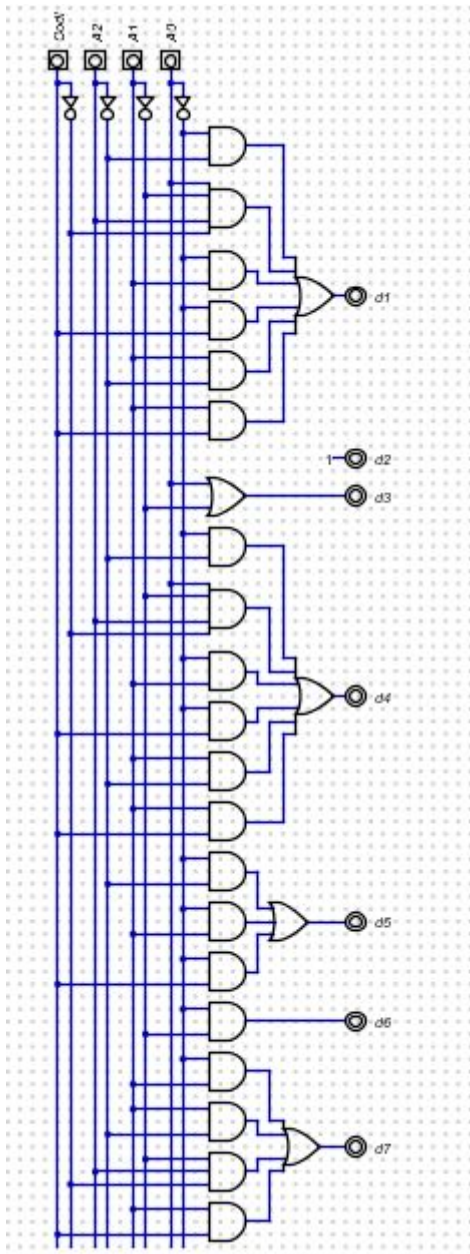
Circuit Resta SM

2.5 Display entrades A i B

Aquest subcircuit els descriurem en conjunt ja que són pràcticament iguals. La funció vendria a ser la mateixa que la del display següent del resultat. Nosaltres haurem elegit un nombre en binari en C2 o SM que vindrà determinat per les variables a_2, a_1, a_0 i b_2, b_1, b_0 . Elegim la variable Codi tota sola sense entrada per estalviar hardware, tenint en compte que aquesta variable compleix amb les condicions de que quan es 1 sempre està en SM i si està a 0, tenim un nombre en C2. La Op només determina si sumam o restam. Les portes OR d'avall serveixen per a unificar totes les variables que hi ha enmig del circuit, és a dir, tenim 4 a_2 , un per a cada operació i elegim el que està activat amb 1, sinó pren el valor de 0. El signe de l'operand ve determinat per a_2/b_2 , ja que si és 1, sigui C2 o SM, sempre sirà un nombre negatiu.



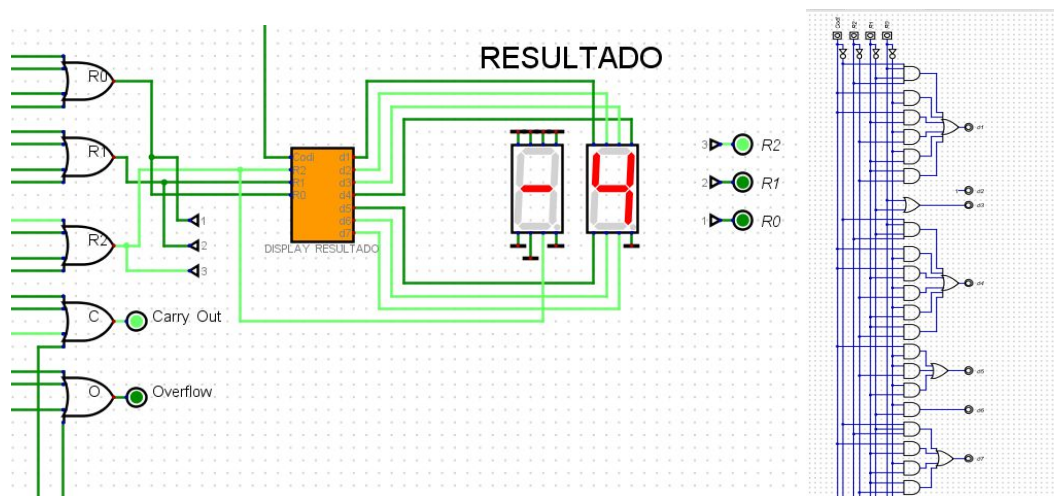
Implementació de ambdós subcircuitos amb els seus displays corresponents



Estructura interna dels subcircuits(són iguals).

2.6 Display resultat

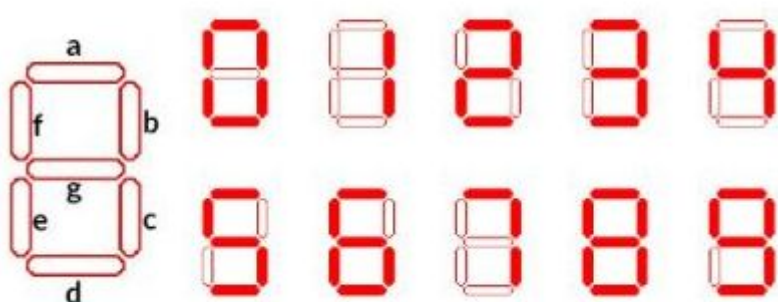
Aquest subcircuit vendria a ser com un convertidor de binari(C2 o SM). Em referesc a que transforma el resultat format per r2,r1 i r0. Ja tenim les entrades en si mateix per a indicar el resultat en C2 o SM, així que utilitzam un display de 7 segments per a mostrar el resultat en decimal. Per a implementar aquest circuit hem creat una taula de veritat que veureu en el punt 3 respectivament que transforma les entrades r2,r1,r0 en d0,d1,d2,d3...(segments del display). El octau segment és el punt, que no utilitzarem.Com podeu veure, el display esquerre només té connectada la entrada r2 al 7 segment. Això és degut a que serà aquest l'únic bit que canviarà el signe.El 8 segment del segon display també és 0 ja que no emplearem el punt decimal.



A la imatge de l'esquerra veim el subcircuit implementat en taronja, a la dreta, la seva estructura interna.

Les entrades del display funcionen així:

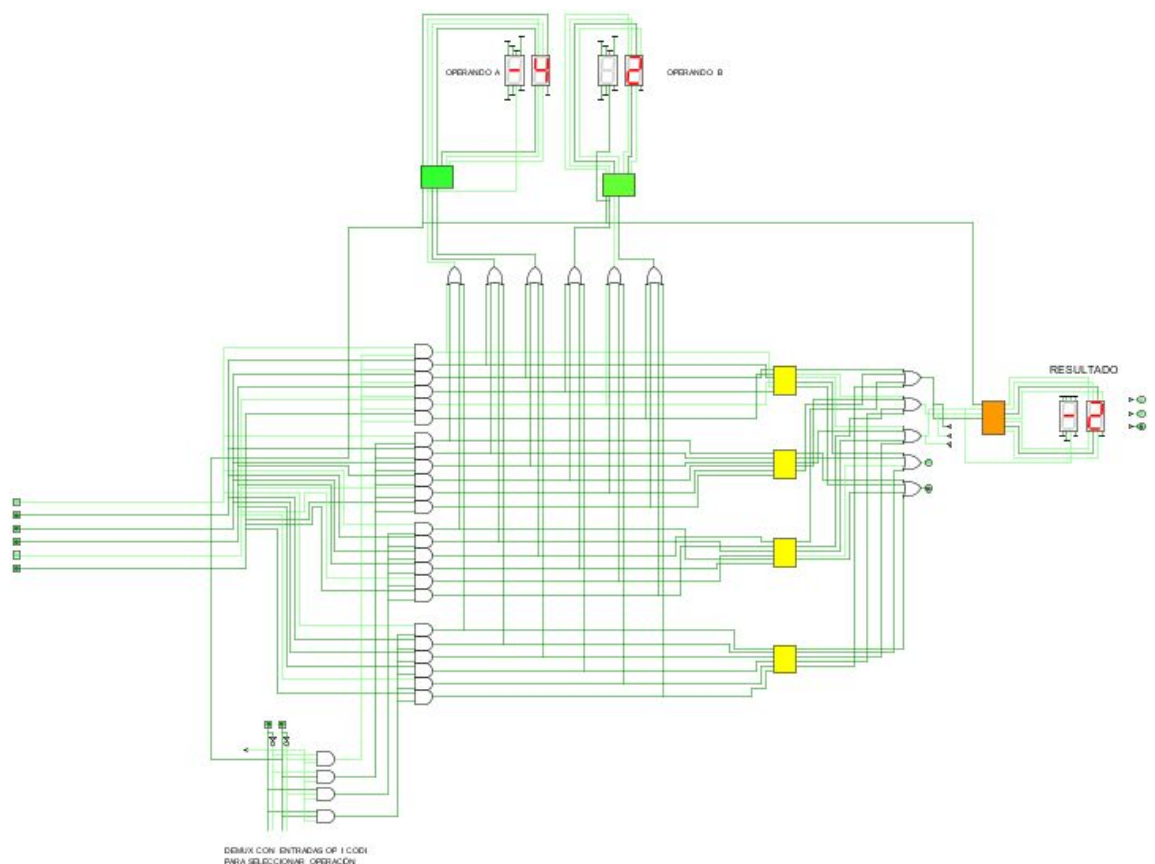
Caracteres del display 7 segmentos



Les lletres van en ordre, essent a la entrada nombre 1, b el nombre 2, c el nombre 3 etc. Veurem això més clar amb la seva respectiva taula de veritat i quins segments s'activen amb cada nombre.

2.7 Demultiplexor

Aquesta part del circuit és la més extensa i que engloba totes les altres. Si recordam la funció d'un multiplexor és la de tenir diverses entrades amb variables de control que determinaran una única sortida (3 bits). Tenim 6 variables d'entrada ($a_2, a_1, a_0/b_2, b_1, b_0$) i dues de control (Op i Codi). Op determina el tipus d'operació i codi si utilitzem C2 o SM ($1=SM, 0=C2$). Com es pot apreciar, codi té connexió amb els displays dels operands i del resultat, ja que s'ha de mostrar aquest nombre en decimal i distingir entre C2 o SM. Dedins el multiplexor tenim els 4 subcircuitos de les operacions integrats i al final tenim portes OR per a determinar el resultat i no tenir més d'una solució, encara que siguin tots 0s. Podem dir que serveixen per a unificar un resultat obtingut amb tots 0s, eliminant sortides i establint com a principal el resultat obtingut. També utilitzam portes AND al principi ja que si elegim una operació amb les entrades de control tindrem com a sortida un 1 en el cable de l'operació corresponent. Per exemple, en aquesta foto veiem com hem elegit la combinació 00 i la línia que va cap aquesta part té el valor de 1, arribant a totes les portes AND de les seves variables.



Circuit complet, implementat en forma de demultiplexor

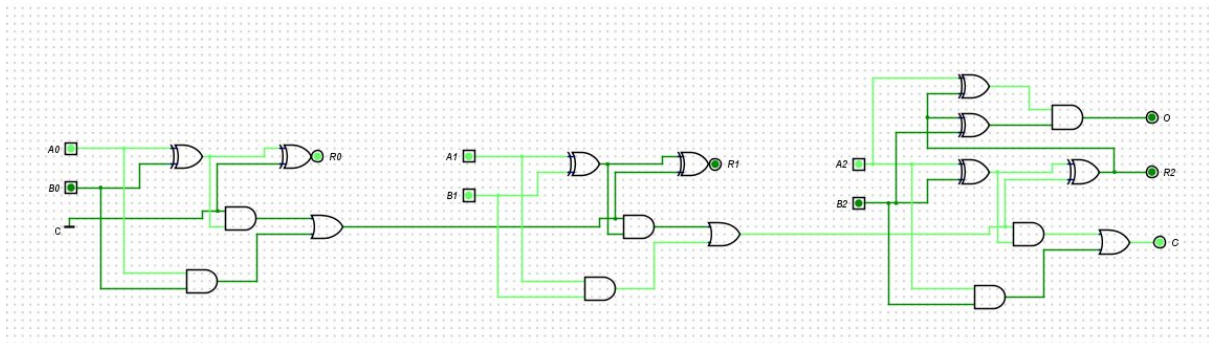
3.Joc de proves

Aquesta part del projecte és molt important ja que ens ajuda a comprovar el correcte funcionament del sistema i els possibles errors que hem haguem pogut fer. Els elements que usarem en aquest apartat siran taules de veritat, exemples etc.

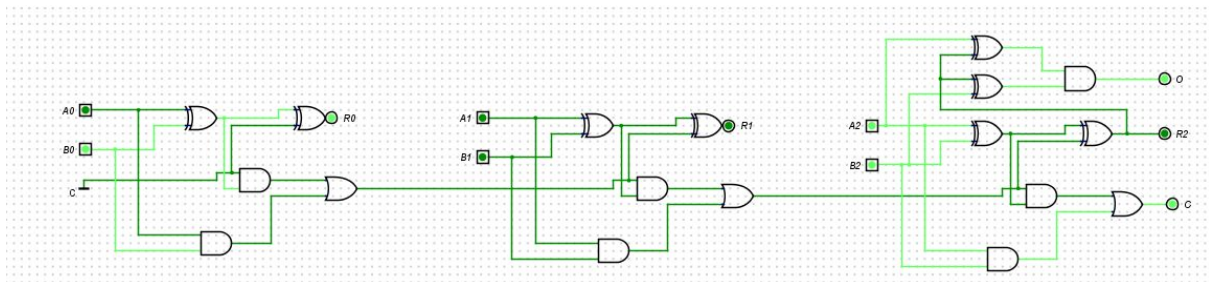
3.1 Suma C2

A0	B0	A1	B1	A2	B2	R0	R1	R2	C	O
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	1	0	0	1	0	0
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	1	1	0	0	0	1	1
0	0	0	1	0	0	0	1	0	0	0
0	0	0	1	0	1	0	1	1	0	0
0	0	0	1	1	0	0	1	1	0	0
0	0	0	1	1	1	0	1	0	1	1
0	0	1	0	0	0	0	1	0	0	0
0	0	1	0	0	1	0	1	1	0	0
0	0	1	0	1	0	0	1	1	0	0
0	0	1	0	1	1	0	1	0	1	1
0	0	1	1	0	0	0	0	1	0	1
0	0	1	1	0	1	0	0	0	1	0
0	0	1	1	1	0	0	0	0	1	0
0	0	1	1	1	1	0	0	1	1	0
0	1	0	0	0	0	1	0	0	0	0
0	1	0	0	0	1	1	0	1	0	0
0	1	0	0	1	0	1	0	1	0	0
0	1	0	0	1	1	1	0	1	0	0
0	1	0	1	0	0	1	1	1	0	0
0	1	0	1	0	1	1	1	1	0	0
0	1	0	1	1	0	1	1	1	0	0
0	1	0	1	1	1	1	1	0	1	1
0	1	1	0	0	0	1	1	0	0	0
0	1	1	0	0	1	1	1	1	0	0
0	1	1	0	1	0	1	1	1	0	0
0	1	1	0	1	1	1	1	0	1	1
0	1	1	1	0	0	1	0	1	0	1
0	1	1	1	0	1	1	0	0	1	0
0	1	1	1	1	0	1	0	0	1	0
0	1	1	1	1	1	1	0	1	1	0
1	0	0	0	0	0	1	0	0	0	0
1	0	0	0	0	1	1	0	1	0	0
1	0	0	0	1	0	1	0	1	0	0
1	0	0	0	1	1	1	0	0	1	1
1	0	0	1	0	0	1	1	0	0	0
1	0	0	1	0	1	1	1	1	0	0
1	0	0	1	1	0	1	1	1	0	0
1	0	0	1	1	1	1	1	0	1	1
1	0	1	0	0	0	1	1	0	0	0
1	0	1	0	0	1	1	1	1	0	0
1	0	1	0	1	0	1	1	1	0	0
1	0	1	0	1	1	1	1	1	0	0
1	0	1	1	0	0	1	0	1	0	1
1	0	1	1	0	1	1	0	0	1	0
1	0	1	1	1	0	1	0	0	1	0
1	0	1	1	1	1	1	0	0	1	0
1	1	0	0	0	0	1	0	1	0	0
1	1	0	0	0	1	0	1	1	0	0
1	1	0	0	1	0	0	1	1	0	0
1	1	0	0	1	1	0	1	0	1	1
1	1	0	1	0	0	0	0	1	0	1
1	1	0	1	0	1	0	0	0	1	0
1	1	0	1	1	0	0	0	0	1	0
1	1	0	1	1	1	0	0	1	1	0
1	1	1	0	0	0	0	0	1	0	1
1	1	1	0	0	1	0	0	0	1	0
1	1	1	0	1	0	0	0	1	1	0
1	1	1	0	1	1	0	0	1	1	0
1	1	1	1	0	0	0	1	1	0	1
1	1	1	1	0	1	0	1	0	1	0
1	1	1	1	1	0	0	1	0	1	0
1	1	1	1	1	1	0	1	1	1	0

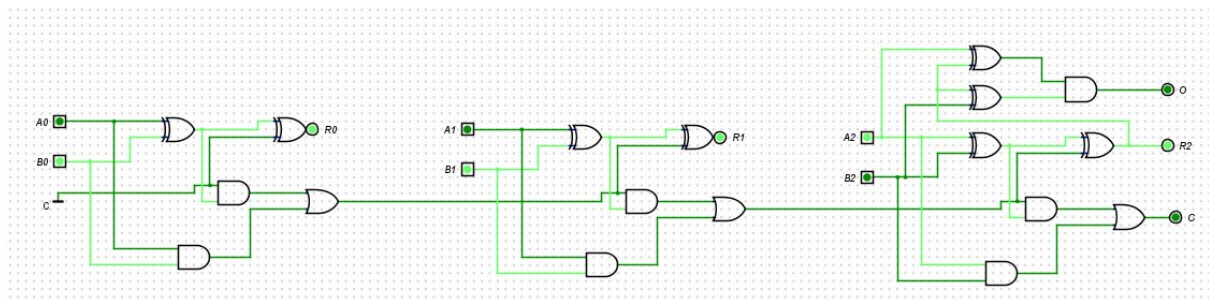
Aquí podem apreciar quant de llarga és aquesta taula de veritat, tinguent 6 variables de entrada i 5 de sortida. El meu joc de proves ha estat revisar aquesta taula de veritat obtinguda amb la opció anàlisi de l'eina Digital. Per a comprovar el seu funcionament també he realitzat algunes proves amb certs exemples.



Aquí podem veure el primer exemple en suma C2. Agafam com a nombre A 111(-1) i li sumam el nombre B 010(+2). $111(-1) + 010(+2) = 001(+1)$ amb carry out=1. No hi ha overflow mai ja que són de diferent signe. En aquest exemple funciona correctament, anem a veure uns altres.



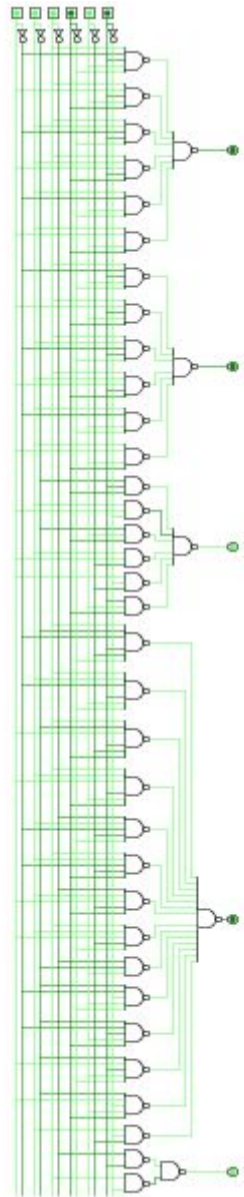
En aquest segon exemple agafarem dos nombres negatius per a veure si podem aconseguir simular una suma amb overflow. $A = 100(-4)$, $B = 101(-3)$. $100(-4) + 101(-3) = 1001(-7)$. És impossible representar aquest nombre amb 3 bits i tenim que carry out=1 i hi haurà overflow ja que són del mateix signe i el resultat no.



En el darrer exemple que veurem sumarem $A = 100(-4)$ i $B = 011(+3)$. El resultat haurà de ser 111(-1) sense carry out ni overflow. Fet aquest joc de proves podem concloure que està ben implementat aquest operació.

3.2 Suma SM

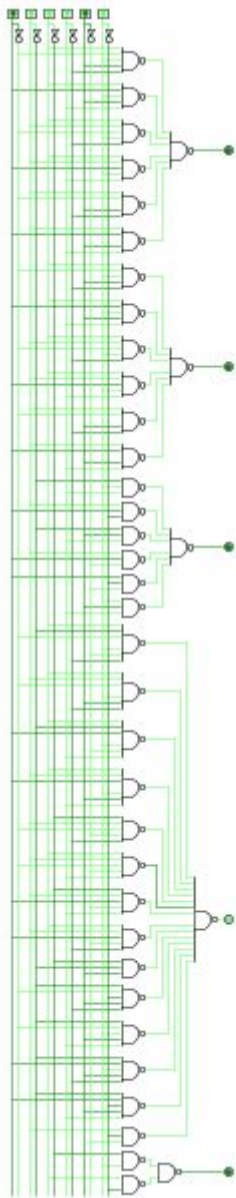
A2	A1	A0	B2	B1	B0	O	C	R2	R1	R0
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	1	0	0	0	0	1
0	0	0	0	1	0	0	0	0	1	0
0	0	0	0	1	1	0	0	0	1	1
0	0	0	1	0	0	0	0	0	0	0
0	0	0	1	0	1	0	0	1	0	1
0	0	0	1	1	0	0	0	1	1	0
0	0	0	1	1	1	0	0	1	1	1
0	0	1	0	0	0	0	0	0	0	1
0	0	1	0	0	1	0	0	0	1	0
0	0	1	0	1	0	0	0	0	1	1
0	0	1	0	1	1	1	1	1	0	0
0	0	1	1	0	0	0	0	0	0	1
0	0	1	1	0	1	0	0	1	0	0
0	0	1	1	1	0	0	0	0	1	0
0	0	1	1	1	1	1	0	0	1	1
0	1	0	0	0	0	0	0	0	1	0
0	1	0	0	1	0	1	1	0	0	0
0	1	0	0	1	1	1	1	0	0	1
0	1	0	1	0	0	0	0	0	0	1
0	1	0	1	0	1	0	0	0	0	0
0	1	0	1	1	0	0	0	1	0	1
0	1	1	0	0	0	0	0	0	1	1
0	1	1	0	0	1	1	1	0	0	0
0	1	1	0	1	0	1	1	0	0	1
0	1	1	0	1	1	1	1	0	1	0
0	1	1	1	0	0	0	0	0	1	1
0	1	1	1	0	0	0	0	0	1	0
0	1	1	1	0	1	0	0	0	1	1
0	1	1	1	1	0	0	0	0	0	1
0	1	1	1	1	1	0	0	1	0	0
1	0	0	0	0	0	0	0	0	1	0
1	0	0	0	0	1	0	0	0	0	1
1	0	0	0	1	0	0	0	0	0	1
1	0	0	0	1	1	0	0	0	1	1
1	0	0	1	0	0	0	0	1	0	0
1	0	0	1	0	1	0	0	1	0	1
1	0	0	1	1	0	0	0	0	1	1
1	0	1	0	0	0	0	0	1	0	1
1	0	1	0	0	1	0	0	0	0	0
1	0	1	0	1	0	0	0	0	0	1
1	0	1	0	1	1	0	0	0	0	1
1	0	1	1	0	0	0	0	0	1	0
1	0	1	1	0	0	1	0	0	1	0
1	0	1	1	0	1	0	0	0	0	1
1	0	1	1	1	0	0	0	0	1	1
1	0	1	1	1	1	1	1	1	0	0
1	1	0	0	0	0	0	0	1	1	0
1	1	0	0	0	1	0	0	1	0	0
1	1	0	0	1	1	0	0	0	0	1
1	1	0	1	0	0	0	0	0	1	1
1	1	0	1	0	1	0	0	1	1	0
1	1	0	1	1	0	0	0	1	1	1
1	1	0	1	1	1	0	1	1	0	0
1	1	1	0	1	1	1	1	1	0	1
1	1	1	1	0	0	0	0	1	1	1
1	1	1	1	0	0	1	0	0	1	0
1	1	1	1	0	1	1	1	1	0	0
1	1	1	1	1	0	1	1	1	0	1
1	1	1	1	1	1	1	1	1	1	0



Tornam a fer el mateix procediment que abans, obtenim la taula de veritat i anam comprovant amb alguns exemples. La taula de veritat és de 6 variables de entrada i 5 de sortida.

A l'exemple adjunt hem elegit sumar $111(-3)+010(+2)$. El resultat és $101(-1)$, sense carry out ni overflow. Com veiem a la imatge, el resultat està bé. L'aparició de les variables en ordre és $Ov=0$, Carry Out=0, $R2=1, R1=0, R0=1$.

Feta la primera comprovació, en farem una altra i procedirem a canviar a l'operació següent.



Segon i darrer exemple per a comprovar que el subcircuit funciona correctament. $A=011(3)$, $B=101(-1)$. El resultat ha de ser $010(+2)$, sense carry out ni overflow.

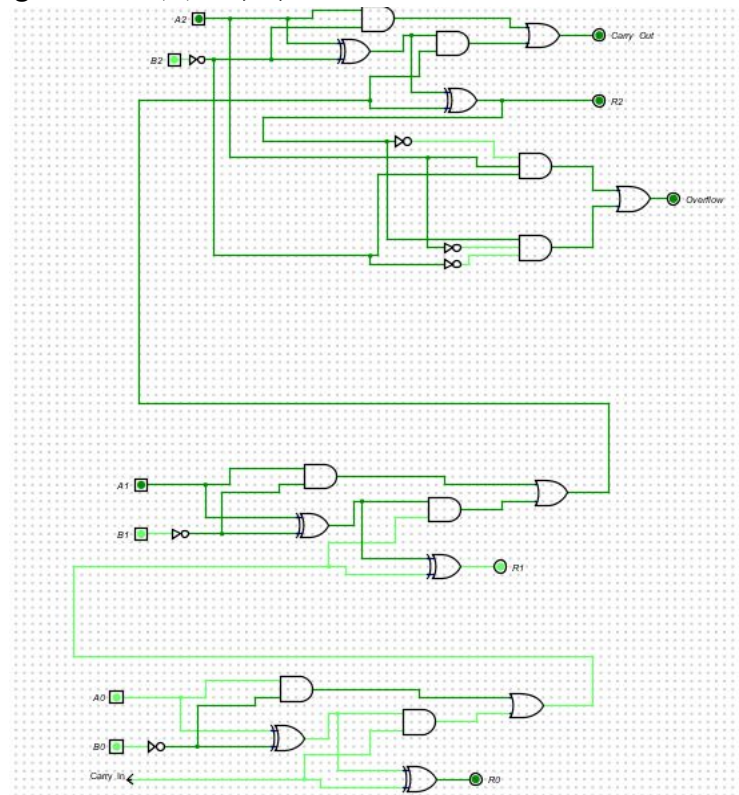
Podem veure clarament que el resultat es mostra com toca, fet que ens fa obtenir la conclusió que aquest circuit funciona correctament.

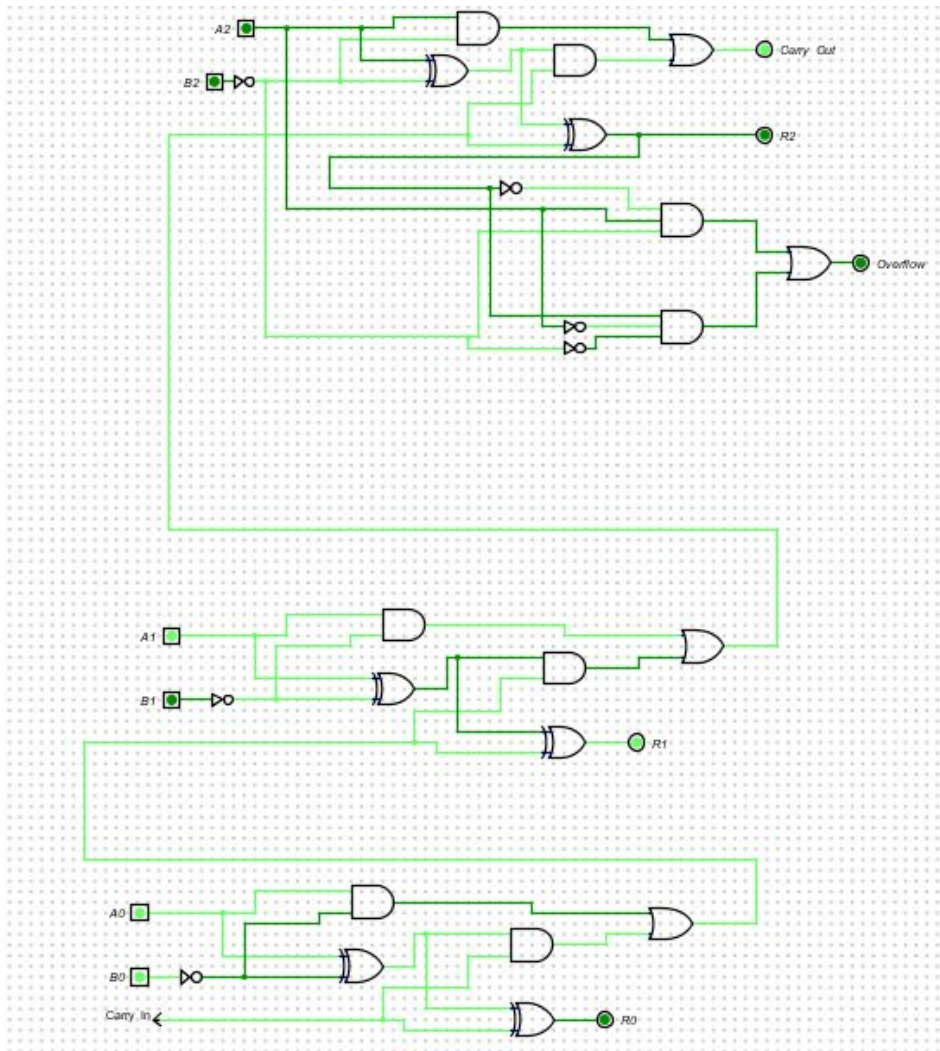
3.3 Resta C2

A0	B0	A1	B1	B2	A2	R0	R1	Carry	R2	Overf
0	0	0	0	0	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	0	1	1	0	0	1	0	0
0	0	0	1	0	0	0	1	0	1	0
0	0	0	1	0	1	0	1	1	0	1
0	0	0	1	1	0	0	1	0	0	0
0	0	0	1	1	1	0	1	0	1	0
0	0	1	0	0	0	0	1	1	1	0
0	0	1	0	0	1	0	1	1	1	0
0	0	1	0	1	1	0	1	1	0	0
0	0	1	1	0	0	0	0	1	0	0
0	0	1	1	0	1	0	0	1	1	0
0	0	1	1	1	0	0	0	1	1	0
0	0	1	1	1	1	0	0	0	1	1
0	0	1	1	1	1	0	0	1	0	0
0	1	0	0	0	0	1	1	0	1	0
0	1	0	0	0	1	1	1	1	0	1
0	1	0	0	1	0	1	1	0	0	0
0	1	0	0	1	1	1	1	0	1	0
0	1	0	0	1	1	1	1	0	1	0
0	1	0	1	0	0	1	1	0	0	0
0	1	0	1	0	1	1	1	0	1	0
0	1	0	1	1	0	1	1	0	1	0
0	1	0	1	1	1	1	1	0	1	0
0	1	1	0	0	0	1	1	0	1	0
0	1	1	0	0	1	1	1	0	1	0
0	1	1	0	1	0	1	1	1	0	1
0	1	1	0	1	1	1	1	1	0	1
1	0	0	0	0	0	1	0	1	0	0
1	0	0	0	0	1	1	0	1	1	0
1	0	0	0	1	0	1	0	0	1	1
1	0	0	0	1	1	1	0	1	0	0
1	0	0	1	0	0	1	1	0	1	0
1	0	0	1	0	1	1	1	0	1	0
1	0	0	1	1	0	1	1	1	0	0
1	0	0	1	1	1	1	1	0	1	0
1	0	1	0	0	0	1	1	1	1	0
1	0	1	0	0	1	1	1	1	0	0
1	0	1	1	0	0	1	1	0	1	1
1	0	1	1	0	1	1	1	0	1	1
1	0	1	1	1	0	1	1	0	1	0
1	1	0	0	0	0	0	0	1	0	0
1	1	0	0	0	1	0	0	1	1	0
1	1	0	0	1	0	0	0	0	1	1
1	1	0	0	1	1	0	0	1	0	0
1	1	0	1	0	0	0	1	0	1	0
1	1	0	1	0	1	0	1	1	0	1
1	1	0	1	1	0	0	1	0	0	0
1	1	0	1	1	1	0	1	0	1	0
1	1	1	0	0	0	0	1	1	0	0
1	1	1	0	0	1	0	1	1	0	0
1	1	1	0	1	0	0	0	1	1	0
1	1	1	0	1	1	0	0	0	1	1
1	1	1	1	0	0	0	0	1	1	0
1	1	1	1	0	1	0	0	0	1	1
1	1	1	1	1	0	0	0	1	0	0

Aquí a l'esquerra som capaços de veure la seva respectiva taula de veritat, on s'indiquen els nombres que s'operen i el resultat, junt al Carry Out i a l'Overflow. Les variables de entrada son 6, b2,b1,b0,a2,a1,a0(cada un dels bits dels nombres A i B).

Feta ja la taula de veritat, anem a comprovar-ho amb una simulació del circuit. Farem la operació A(001,1 en decimal) menys B(111,-1 en decimal) sirà igual a 010(2). 1-(-1)=2





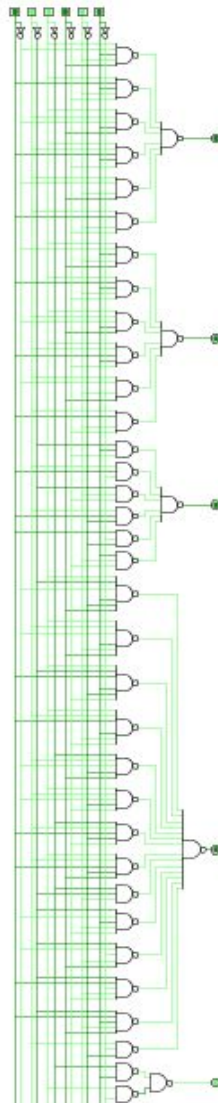
Darrer exemple: $A=011(3)$, $B=001(1)$. Resultat= $010(2)$, amb Carry Out=1. Podem comprovar que funciona amb normalitat.

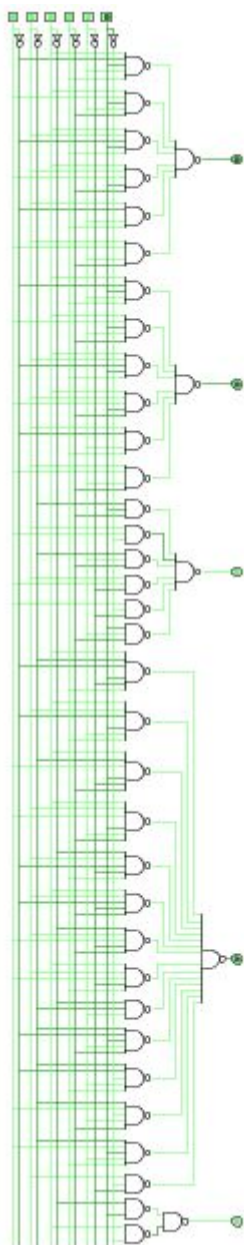
3.4 Resta SM

A2	A1	A0	B2	B1	B0	O	C	R2	R1	R0
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	1	0	0	0	1	1	0
0	0	0	0	1	1	0	0	1	1	1
0	0	0	1	0	0	0	0	0	0	0
0	0	0	1	0	1	0	0	0	0	1
0	0	0	1	1	0	0	0	0	0	1
0	0	0	1	1	1	0	0	0	1	1
0	0	1	0	0	0	0	0	0	0	1
0	0	1	0	0	1	0	0	0	1	0
0	0	1	0	1	0	0	0	1	0	1
0	0	1	0	1	1	0	0	1	0	0
0	0	1	1	0	0	0	0	0	0	1
0	0	1	1	0	1	0	0	0	1	0
0	0	1	1	1	0	0	0	0	1	1
0	0	1	1	1	1	0	0	0	1	1
0	1	0	0	0	0	0	0	0	1	0
0	1	0	0	0	1	0	0	0	0	1
0	1	0	0	1	0	0	0	0	0	0
0	1	0	0	1	1	0	0	1	0	1
0	1	0	1	0	0	0	0	0	1	0
0	1	0	1	0	1	0	0	0	1	1
0	1	0	1	1	0	1	1	0	0	0
0	1	0	1	1	1	1	1	0	0	1
0	1	1	0	0	0	0	0	0	1	1
0	1	1	0	0	1	0	0	0	1	0
0	1	1	0	1	0	0	0	0	0	1
0	1	1	0	1	1	0	0	1	0	0
0	1	1	1	0	0	0	0	0	1	0
0	1	1	1	0	1	0	0	0	0	1
0	1	1	1	1	0	1	1	0	0	0
0	1	1	1	1	1	1	1	0	1	0
1	0	0	0	0	0	0	0	1	0	0
1	0	0	0	0	1	0	0	1	0	1
1	0	0	0	1	0	0	0	1	1	0
1	0	0	0	1	1	0	0	1	1	1
1	0	0	1	0	0	0	0	1	0	0
1	0	0	1	0	1	0	0	0	0	1
1	0	0	1	1	0	0	0	0	1	0
1	0	0	1	1	1	0	0	0	1	1
1	0	1	0	0	0	0	0	1	0	1
1	0	1	0	0	1	0	0	1	1	0
1	0	1	0	1	0	0	0	1	1	1
1	0	1	0	1	1	1	1	1	0	0
1	0	1	1	0	0	0	0	1	0	1
1	0	1	1	0	1	0	0	0	0	0
1	0	1	1	1	0	0	0	0	0	1
1	0	1	1	1	1	0	0	0	1	0
1	1	0	0	0	0	0	0	1	1	0
1	1	0	0	0	1	0	0	1	1	1
1	1	0	0	1	0	1	1	1	0	0
1	1	0	0	1	1	1	1	1	0	1
1	1	0	1	0	0	0	0	1	1	0
1	1	0	1	0	1	0	0	1	0	1
1	1	0	1	1	0	0	0	0	0	1
1	1	1	0	0	0	0	0	1	1	1
1	1	1	0	0	1	1	1	1	0	0
1	1	1	0	1	0	1	1	1	0	1
1	1	1	0	1	1	0	0	1	1	0
1	1	1	1	0	0	0	0	1	0	1
1	1	1	1	0	1	0	0	0	1	0
1	1	1	1	1	0	0	0	0	0	0

Aquesta és la darrera taula de veritat de les 4 operacions que tenim en tot el circuit. És similar a les altres, amb les mateixes variables d'entrada i de sortida. Ara anem a fer algunes simulacions amb el circuit.

Com podeu apreciar, farem la operació **011-010=001**. No hi ha d'haver cap tipus de carry out ni de overflow. Això es compleix en aquest circuit.



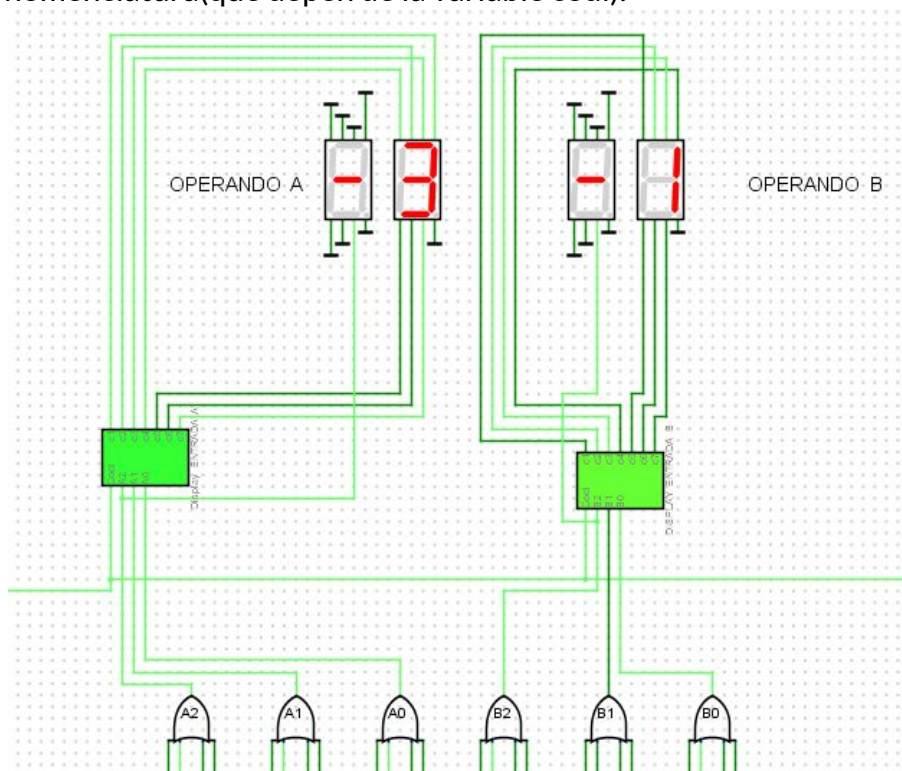


Com a segona i darrera prova del subcircuit que ens resta en SM farem la operació **111(-3)-110(-2)=101(-1)**. Podem apreciar que el resultat s'indica correctament sense overflow ni carry out.

3.5 Display entrades A,B

Codi	A2	A1	A0	d1	d2	d3	d4	d5	d6	d7
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	1	1	1	0	0	1
0	1	1	0	1	1	0	1	1	0	1
0	1	1	1	0	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	0
1	0	0	1	0	1	1	0	0	0	0
1	0	1	0	1	1	0	1	1	0	1
1	0	1	1	1	1	1	1	0	0	1
1	1	0	0	1	1	1	1	1	1	0
1	1	0	1	0	1	1	0	0	0	0
1	1	1	0	1	1	0	1	1	0	1
1	1	1	1	1	1	1	1	0	0	1

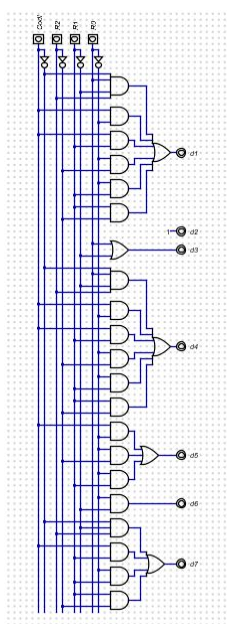
Aquesta vendria a ser la taula de veritat del display entrada A. La del display d'entrada B és pràcticament la mateixa amb els noms de variables d'entrada canviats. Aquesta taula ens activa els segments que corresponen a cada nombre, determinat per la seva nomenclatura (que depèn de la variable codi).



Exemple: A=111, B=101 SM.

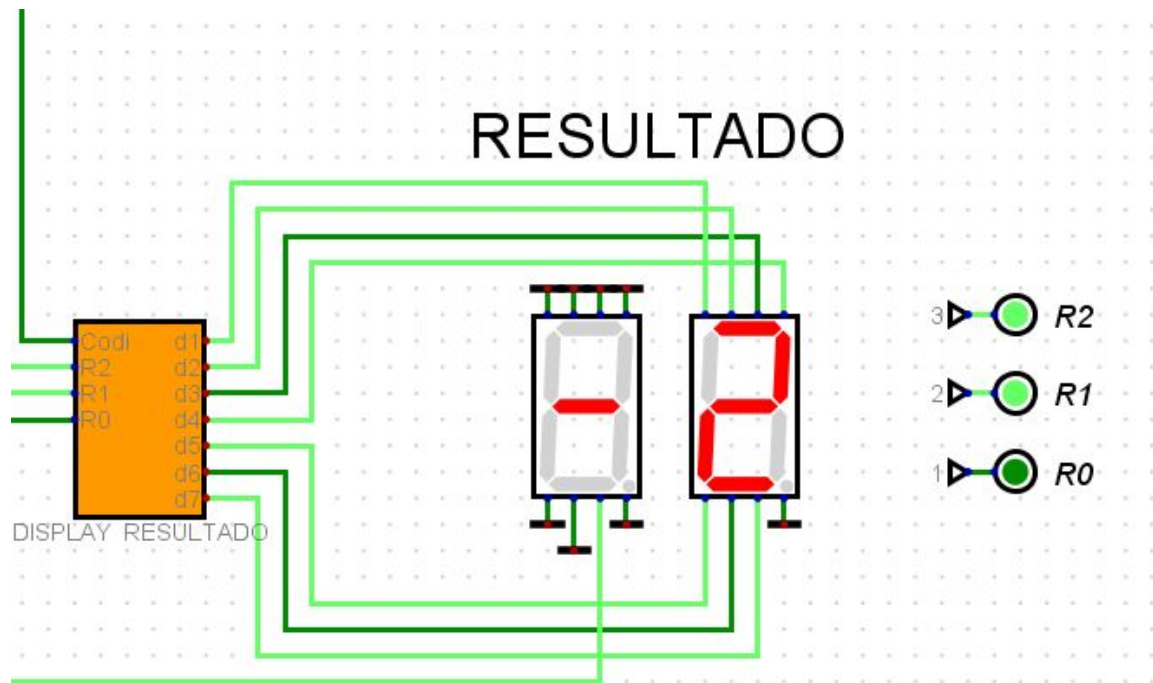
3.6 Display resultat

Codi	R2	R1	R0	d1	d2	d3	d4	d5	d6	d7
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	1	1	1	0	0	1
0	1	1	0	1	1	0	1	1	0	1
0	1	1	1	0	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	0
1	0	0	1	0	1	1	0	0	0	0
1	0	1	0	1	1	0	1	1	0	1
1	0	1	1	1	1	1	1	0	0	1
1	1	0	0	1	1	1	1	1	1	0
1	1	0	1	0	1	1	0	0	0	0
1	1	1	0	1	1	0	1	1	0	1
1	1	1	1	1	1	1	1	0	0	1



A l'esquerra, taula de veritat. A la dreta, la seva estructura interna.

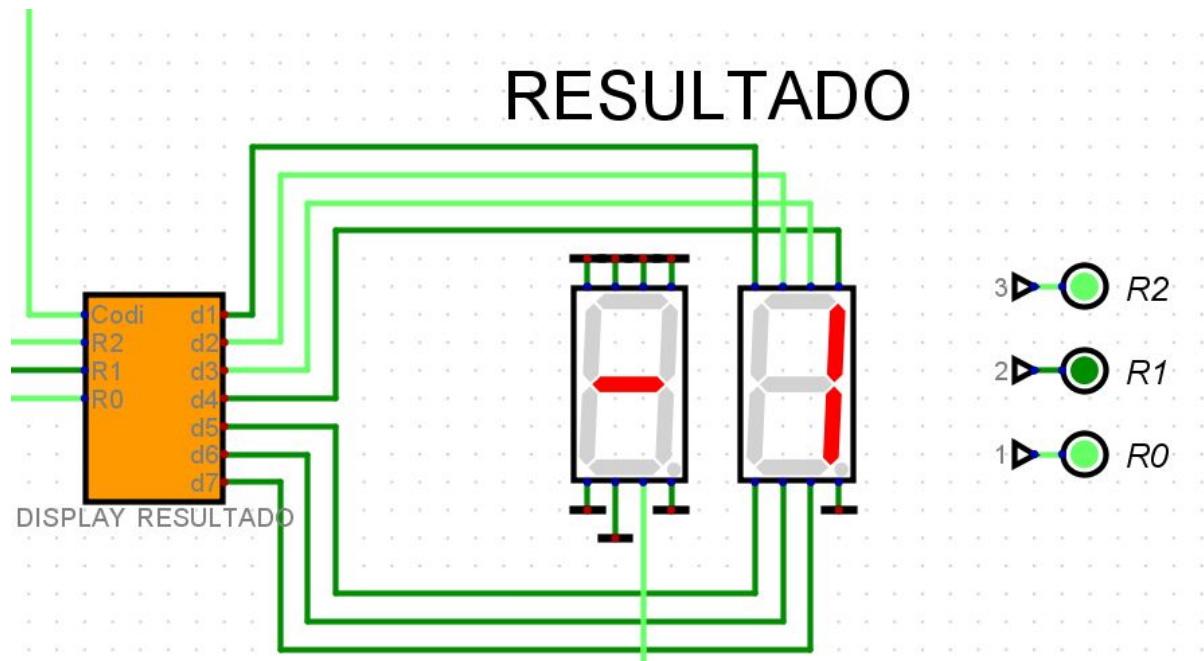
Com podeu veure, les 3 taules de veritat són gairebé iguals, just canviant el nombre de les variables de entrada per A,B o R, la lletra amb la qual representam cada nombre. Per tant, si les taules de veritat són iguals, els circuits també seran iguals, just amb la diferència de la nomenclatura de les variables.
Anem a veure alguns exemples:



Representació de 110 en C2(-2), els segments 1,2,4,5,7 s'activen. El display de l'esquerra representa el signe negatiu ja que R2=1.

Codi	R2	R1	R0	d1	d2	d3	d4	d5	d6	d7
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	1	1	1	0	0	1
0	1	1	0	1	1	0	1	1	0	1

Mireu les entrades 0110 i veureu com s'activen els segments d1,d2,d4,d5,d7.

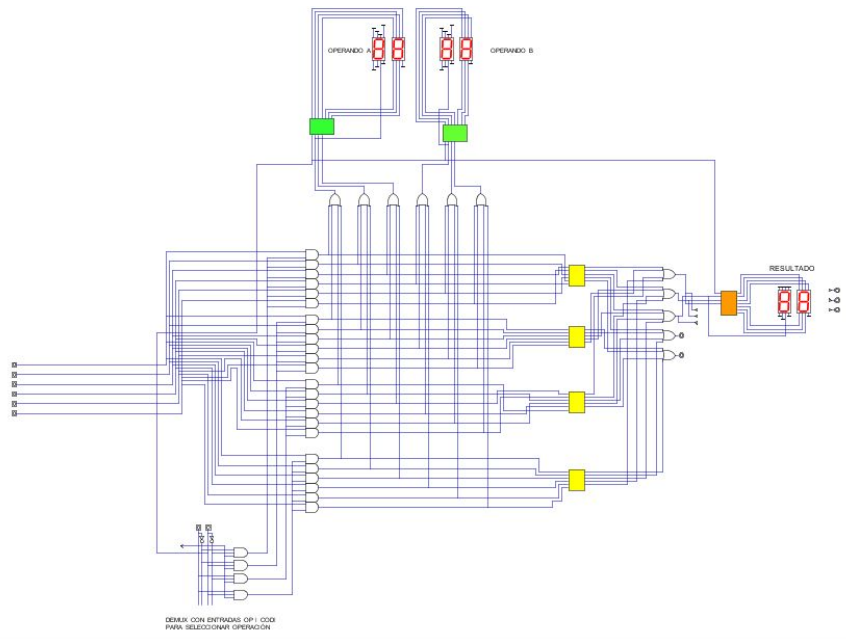


Representació del nombre 101 en SM(-1). El display de l'esquerra representa el nombre en negatiu ja que R2=1.

Codi	R2	R1	R0	d1	d2	d3	d4	d5	d6	d7
1	1	0	1	0	1	1	0	0	0	0
1	1	1	0	1	1	0	1	1	0	1
1	1	1	1	1	1	1	1	0	0	1

Mireu les entrades 1101 i com s'activen els segments d2,d3

3.7 Circuit complet



Això és com es veu el circuit complet, a l'esquerra tenim les variables d'entrada(a2,b1,a0...) que juntament amb les variables de control, passaran per portes AND, per tal de implementar un demultiplexor i elegir un dels 4 subcircuits que representa una sola operació amb C2 o SM. Els displays d'adalt mostraran els nombres introduïts en forma decimal, depenent del tipus de binari elegit. Finalment, veurem el resultat de l'operació corresponent a la combinació OpCodi en el display de la dreta del tot.

-Prova 1(suma C2):

OpCodi=00;

A=111(-1) + B=011(+3) = C

C=010(+2)

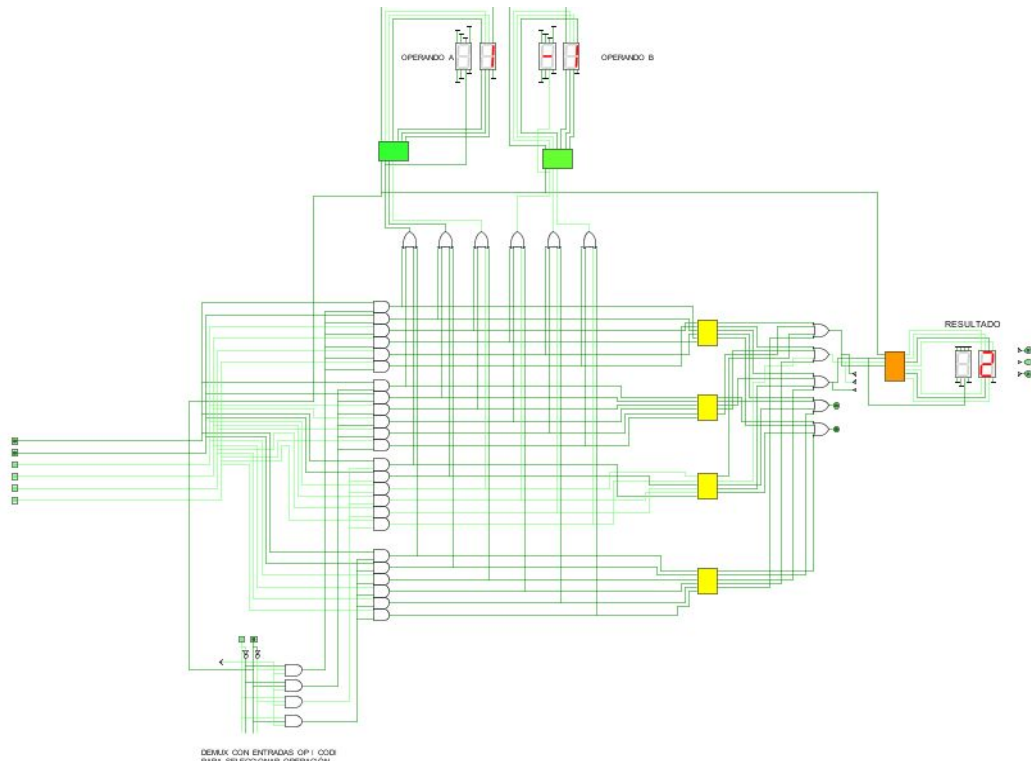
-Prova 3(resta C2):

OpCodi=10

A=001(+1) B=111(-1)

001-111=010

C=010

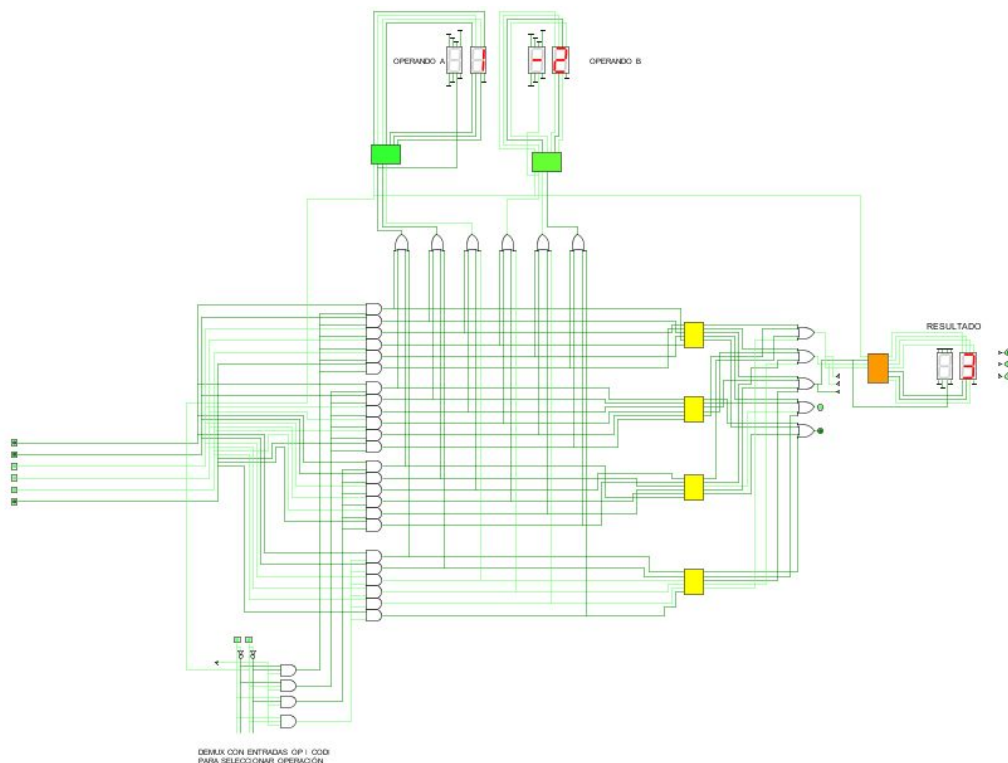


Resta C2; A(001)-B(111)=1-(-1)=2(010), Cout=0, Ov=0.

-Prova 4(resta SM):

OpCodi=11

A=001,B=110



Resta SM; 001-110=010,Cout=0,Ov=0.

4.Conclusió

Després d'aquesta pràctica m'he adonat compte de la importància de les portes lògiques en la implementació dels aparells que usam en la quotidianitat. El circuit que hem implementat podria ser una petita part de una calculadora o inclús crear un sistema que representés un marcador. L'electrònica està molt present en el nostre dia a dia i hem après un poc la seva estructura interna més bàsica. A part d'això, també m'ha agradat crear aquest circuit ja que hi veig una similitud amb programar. El fet de analitzar mentalment quina és la millor opció per a resoldre un problema i de la manera més eficient possible ens fa treballar molt la nostra capacitat analítica. He de dir que estic content amb el resultat i he tingut bones sensacions.

PD: hi ha un petit error al circuit, concretament en la sortida de carry out en la resta C2. En alguns casos surt 1 sense motiu i no ho he sabut arreglar.

Gràcies,

Jaume